

SymPy Bug Report

1 Bug 41: Unsound Extended-Real Assumption for a Reciprocal

1.1 Minimal Reproducer

```
from sympy import symbols
from sympy.assumptions import Q, ask

xr = symbols("xr", real=True)
expr = 1/(xr - 1)

print("ask(Q.extended_real(expr)) =", ask(Q.extended_real(expr)))
print("expr.subs(xr, 1) =", expr.subs(xr, 1))
print("substituted value is_extended_real =", expr.subs(xr, 1).is_extended_real)
```

1.2 Observed Behavior

```
ask(Q.extended_real(expr)) = True
expr.subs(xr, 1) = zoo
substituted value is_extended_real = False
```

SymPy proves that $1/(xr - 1)$ is extended real for an arbitrary real symbol xr . This is unsound: $xr = 1$ is an allowed real substitution, and at that point the expression evaluates to `zoo`, whose `is_extended_real` property is `False`.

1.3 Expected Behavior

The query `ask(Q.extended_real(1/(xr - 1)))` should not return `True` under only the assumption that xr is real. The correct result is `False` or `None`; in either case, SymPy should not prove the extended-real predicate without knowing that $xr - 1 \neq 0$.

1.4 Mathematical Explanation

The extended real line consists of real values together with the signed infinities $-\infty$ and $+\infty$. It does not include complex infinity `zoo`. For

$$f(xr) = \frac{1}{xr - 1},$$

the denominator vanishes at the admissible real value $xr = 1$. Therefore

$$f(1) = \frac{1}{0} = \text{zoo}, \quad \text{zoo} \notin \mathbb{R} \cup \{-\infty, +\infty\}.$$

A universally valid assumptions conclusion for a real symbol must be compatible with every substitution satisfying that symbol assumption. Since the substitution $xr = 1$ contradicts the claimed predicate, the result is not a harmless simplification or merely an unknown singular point: it is a proof of a property that the expression does not have over its full declared real domain.

1.5 Source-Level Diagnosis

The diagnosis locates the relevant rule in `sympy/assumptions/handlers/sets.py`, in the extended-real predicate handler registered for `Add`, `Mul`, and `Pow`. The traced public call asks the extended-real predicate for $(xr - 1)^{-1}$. The expression is a `Pow` whose arguments are $xr - 1$ and -1 . The handler delegates to `test_closed_group` in `sympy/assumptions/handlers/common.py`; that helper checks the predicate on each argument and returns `True` when all arguments are extended real. The recorded trace confirms that the base $xr - 1$ is accepted as extended real and then the whole reciprocal power is accepted:

```
expr.args -> (xr - 1, -1)
ask(Q.extended_real(xr - 1)) -> True
ask(Q.extended_real((xr - 1)**-1)) -> True
```

The problematic rule is:

```
@ExtendedRealPredicate.register_many(Add, Mul, Pow)
def _(expr, assumptions):
    return test_closed_group(expr, assumptions, Q.extended_real)
```

This closed-group reasoning is too broad for `Pow`. In particular, a negative exponent creates a reciprocal, and the reciprocal of an extended-real quantity is extended real only when the base is known to be nonzero. Here $xr - 1$ is extended real but can be zero, so the handler proves the reciprocal extended real while omitting the necessary side condition. The diagnosis suggests splitting `Pow` away from the simple `Add/Mul` closed-group rule and requiring the base to be provably nonzero before returning `True` for negative exponents or reciprocals; otherwise the handler should return an unknown result.

1.6 Additional Failing Instances

The independent verification covers the same failure pattern for shifted simple poles:

$$\frac{1}{xr - a}, \quad a \in \{1, 0, -1, 2, 3, -2\}.$$

In every case xr is a real symbol, SymPy returns `True` for `ask(Q.extended_real(1/(xr - a)))`, and substituting the real value $xr = a$ gives `zoo`. The expected behavior is therefore the same across the family: without a proof that the denominator is nonzero, SymPy should not prove the expression extended real.

Input / Expression	SymPy behavior	Expected behavior
$1/(xr - 1), xr = 1$	True; value zoo	not True
$1/xr, xr = 0$	True; value zoo	not True
$1/(xr + 1), xr = -1$	True; value zoo	not True
$1/(xr - 2), xr = 2$	True; value zoo	not True
$1/(xr - 3), xr = 3$	True; value zoo	not True
$1/(xr + 2), xr = -2$	True; value zoo	not True

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest public material identified was SymPy's assumptions predicate documentation for `Q.extended_real`, which confirms that the extended reals are real numbers together with signed infinities, and documentation for the corresponding `ExtendedRealPredicate` machinery. No public issue was found for the exact query `ask(Q.extended_real(1/(xr - 1)))` returning `True`, nor for this specific reciprocal-with-possible-zero-denominator counterexample with a symbolic base. The same root-cause family is, however, publicly tracked: the assumptions handlers unsoundly proving real-ness of a reciprocal with a possibly-zero denominator. This is recorded as issue [#28150](#) (open) — `ask(Q.real(Pow(0, -1, evaluate=False)))` gives `True` — with fix attempts in PRs [#28164](#) (closed) and [#28377](#) (open). The deduplication verdict is therefore a known family with a specific new manifestation rather than a wholly novel bug. This `Q.extended_real` symbolic-base instance is nonetheless distinct and still unfixed: those PRs only touch a concrete-number base through the `Q.real` number handler, and on the current release `ask(Q.real(1/(xr - 1)))` already returns `None` while only `Q.extended_real` is wrongly `True`, so they would not resolve it. The report remains individually actionable because it gives an exact counterexample, a traced handler location, and a focused regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import zoo, symbols
from sympy.assumptions import Q, ask

@pytest.mark.parametrize("a", [1, 0, -1, 2, 3, -2])
def test_extended_real_reciprocal_requires_nonzero_denominator(a):
    xr = symbols("xr", real=True)
    expr = 1/(xr - a)
    assert ask(Q.extended_real(expr)) is not True
    assert expr.subs(xr, a) == zoo
```